

CRUD

Django y Pydantic



18/05/2025 - IA - Gemini

Creación de CRUD en Django con Pydantic

Prompts

ACTUA COMO UN PROGRAMADOR EXPERTO, CREA UN CRUD Y LOS ARCHIVOS NECESARIOS PARA QUE UTILICE LA ESTRUCTURA DE DJANGO MODEL TEMPLATE VIEW (MTV), CREA LOS MODELS, TEMPLATES, VIEW, URLS, FORMS Y APP CORRESPONDIENTES, USARAS COMO BASE DE DATOS POSTGRES. REALIZA EL CONTROL DE LAS VALIDACIONES CON PYDANTIC A TRAVES DE MENSAJES DE ERROR PERSONALIZADOS USANDO DJANGO, LOS MENSAJES DE ERROR SE DEBEN MOSTRAR USANDO LA LIBRERÍA SWEETALERT2

//

EL NOMBRE DE LA CARPETA QUE CONTENDRA MI PROYECTO SERA "mis_empleados"

//

EL NOMBRE DEL ENTORNO VIRTUAL SERA "empleados_dj521"

//

EL NOMBRE DE LA BASE DE DATOS SERA "db_empleados_521"

USAR VARIABLES DE ENTORNO PARA LAS CREDENCIALES DE LA BASE DE DATOS

//

EL NOMBRE DEL PROYECTO SERA "form_empleados"

//

CREA EL PROYECTO SIN QUE SE GENERE UNA SUBCARPETA

//

EL NOMBRE DE MIS APP SERAN "empleados" y "usuarios"

//

ESTOS SON LOS DATOS QUE SE PODRAN CREAR, LEER, ACTUALIZAR Y ELIMINAR

//

NOMBRES

APELLIDOS

EDAD

FECHA

EMAIL

TELEFONO_CASA

TELEFONO_MOVIL

SUELDO BASE

COMISION

SUELDO BRUTO

PASSWORD

FOTO

//

CADA INPUT TENDRA LAS SIGUIENTES VALIDACIONES

//

NOMBRES, DEBE TENER UN MAXIMO DE 60 CARACTERES.

APELLIDOS, DEBE TENER UN MAXIMO DE 60 CARACTERES.

EDAD, NO DEBE SER MAYOR DE 100 AÑOS.

FECHA, SE DEBE INGRESAR EN FORMATO DD-MM-AAA.

EMAIL, DEBE TENER FORMATO CORRECTO DE CORREO ELECTRONICO.

TELEFONO_CASA, DEBERA USAR UN FORMATO DE TELEFONO FIJO DE ESPAÑA.

TELEFONO_MOVIL, DEBERA USAR UN FORMATO DE TELEFONO MOVIL DE ESPAÑA.

SUELDO BASE, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

COMISION, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

SUELDO BRUTO, DEBE TENER UN FORMATO PARA SEPARADOR DE MILES Y CENTIMOS.

SUELDO BRUTO, SE DEBE CALCULAR Y ACTUALIZAR AUTOMATICAMENTE CON LA SUMA DE SUELDO BASE + COMISION.

PASSWORD, DEBE TENER COMO MINIMO 8 CARACTERES.

FOTO, SIEMPRE SE DEBE CARGAR UNA IMAGEN,

//

FOTO, COLOCAR UN BOTON QUE PERMITA SELECCIONAR Y CARGAR UNA IMAGEN

//

TODOS LOS INPUT SON DE INGRESO DE DATOS OBLIGATORIO

//

COLOCA UN ICONS EN EL CAMPO PASSWORD PARA VISUALIZAR U OCULAR LO INGRESADO

//

IMPLEMENTA EL HASHING DE CONTRASEÑAS. USA LAS FUNCIONES DE HASHING QUE DJANGO PROPORCIONA (DJANGO.CONTRIB.AUTH.HASHERS)

//

AÑADE AUTENTICACIÓN DE USUARIOS. CREA MODELOS DE USUARIO PERSONALIZADOS (USA EL USER DE DJANGO), FORMULARIOS DE REGISTRO E INICIO DE SESIÓN, Y VISTAS PARA MANEJAR LA AUTENTICACIÓN.

//

AÑADE UN PANEL DE ADMINISTRACIÓN PERSONALIZADO PARA GESTIONAR LOS EMPLEADOS.

//

PARA QUE EL PROYECTO SEA RESPONSIVE UTILIZA BOOTSTRAP V5.0

//

UTILIZA LAS LIBRERIAS BOOTSTRAP V5.0, SWEETALERT2 Y FONT AWESOME FREE DESDE LA NUBE.

//

AGREGA ICONS DE LA LIBRERÍA **FONT AWESOME FREE** A LOS BOTONES DE CREAR, EDITAR Y ELIMINAR ASI COMO A LOS MENSAJES DE ERROR

//

PARA INSTALAR LAS DEPENDENCIAS CREA UN ARCHIVO, Y PONLE DE NOMBRE "requirements.txt".

AGREGA ESTE PAQUETE "python-dotenv" Y HAZ SU IMPLEMENTACIÓN EN EL "settings.py"

//

MUESTRAME EL CONTENIDO DEL ARCHIVO .env

//

MEJORA LA INTERFAZ DE USUARIO: AUNQUE YA ESTAS USANDO BOOTSTRAP 5, EXPLORA FORMAS DE MEJORAR AÚN MÁS LA INTERFAZ DE USUARIO, COMO PAGINACIÓN PARA LISTAS LARGAS, ESTILOS PERSONALIZADOS ADICIONALES, O EL USO DE JAVASCRIPT PARA INTERACCIONES MÁS DINÁMICAS (APARTE DEL TOGGLE DE LA CONTRASEÑA Y SWEETALERT)

//

IMPLEMENTA LA ELIMINACIÓN SUAVE (SOFT DELETE): EN LUGAR DE ELIMINAR PERMANENTEMENTE LOS REGISTROS, IMPLEMENTA UNA ELIMINACIÓN SUAVE, DONDE SE MARCA UN REGISTRO COMO ELIMINADO PERO PERMANECE EN LA BASE DE DATOS.

//

ESPECIFICA EL NOMBRE DE CADA ARCHIVO QUE SE VAYA A UTILIZAR PARA CREAR LO SOLICITADO.

//

EL CONTENIDO DE LOS CODIGOS CSS Y JS DEBEN ESTAR SEPARADOS POR ARCHIVOS Y CARPETAS.

//

MUESTRAME LA ESTRUCTURA DE ARCHIVOS.

//

AL FINALIZAR COLOCA EL TITULO "PROYECTO DJANGO Y PYDANTIC - EXPLICACION DETALLADA" Y MUÉSTRAME EN DETALLE Y PASO A PASO TODOS LOS ARCHIVOS COMPLETOS Y SU RESPECTIVA EXPLICACIÓN PARA INSTALAR FRAMEWORK, LIBRERÍAS Y ENTORNOS ASÍ COMO LA EXPLICACIÓN DE CADA ARCHIVO. MUESTRALO EN ESTE ORDEN :

1 - Estructura de Archivos del Proyecto

2 - Instalación y Configuración

- Crear un Entorno Virtual
- Muéstrame el contenido del archivo requirements.txt.
- Instalar Django, Psycopg2 (para PostgreSQL), Pydantic y todos los paquetes y dependencias que sean necesarios para este proyecto.
- Configurar la Base de Datos PostgreSQL
- Crear el Proyecto y las Aplicaciones
- Añadir las Aplicaciones a settings.py
- Crear y Aplicar las Migraciones
- Crear un Superusuario (para el panel de administración)
- Ejecutar el Servidor de Desarrollo

3 - Contenido de los Archivos

4 - Documentación del Proyecto

GENERA LA DOCUMENTACIÓN PARA EL PROYECTO, EXPLICANDO SU ESTRUCTURA, LOS MODELOS, LAS VISTAS, LOS FORMULARIOS, ETC.

//

Resultado



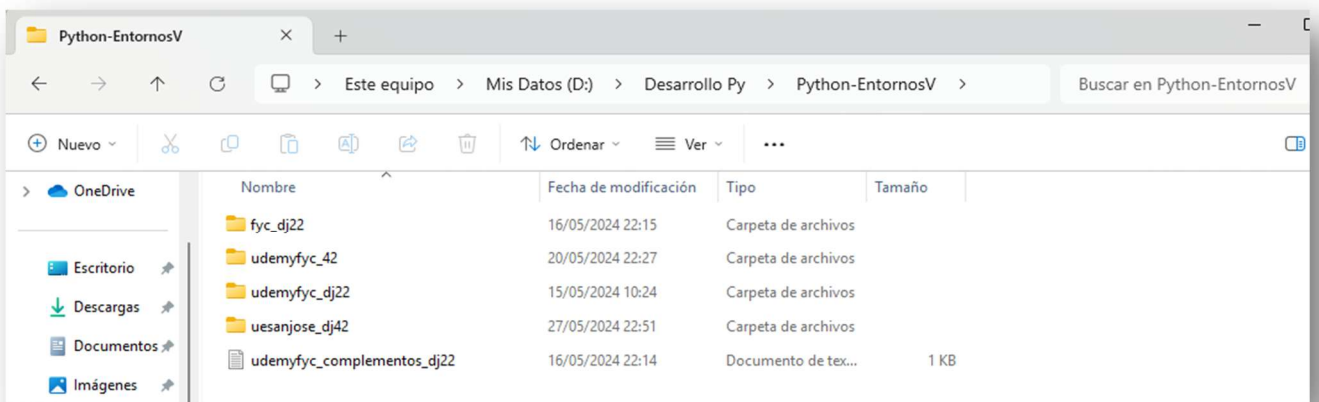
Como programador experto, te guiaré a través de la creación de un CRUD completo en Django utilizando la arquitectura MTV, con PostgreSQL como base de datos y validaciones con Pydantic y SweetAlert2.

(Este documento fue adaptado en base a la respuesta de la IA, el texto completamente generado se identifica con [IA](#))

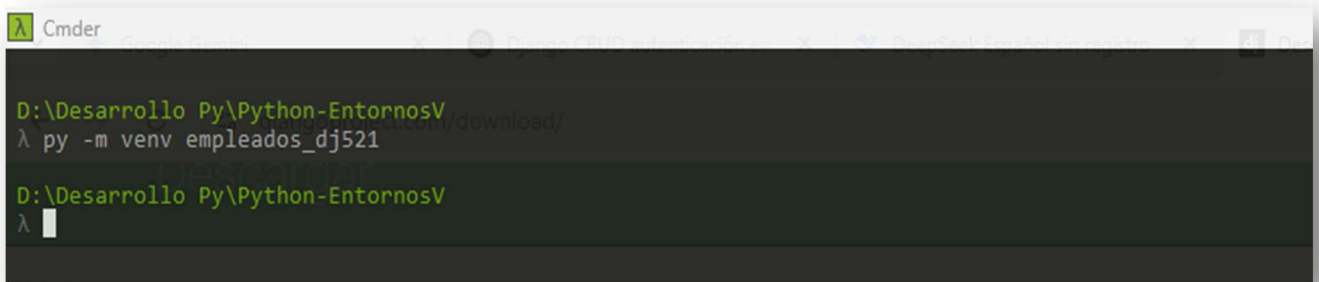
CONFIGURACION INICIAL

CREAR Y ACTIVAR ENTORNO VIRTUAL

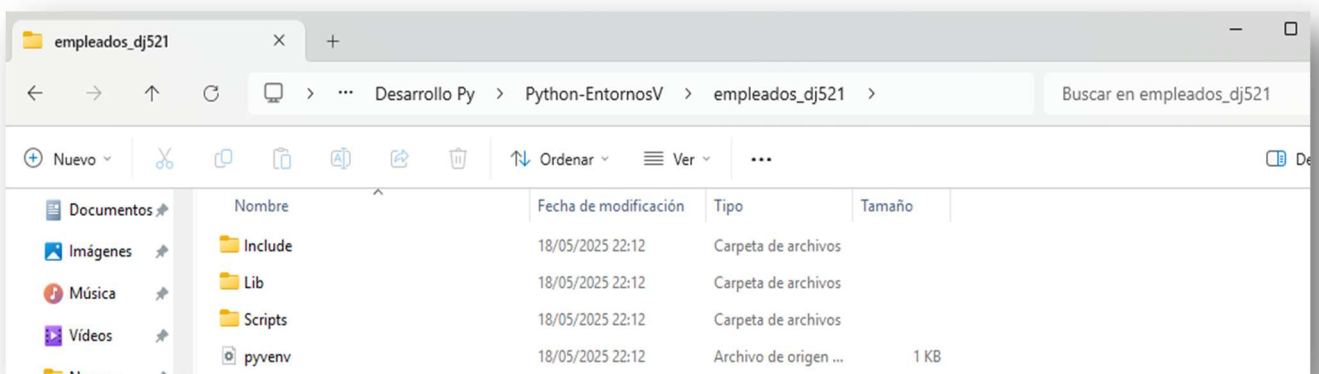
Para ello ingresamos a la carpeta donde tenemos todos nuestros entornos virtuales creados y abrimos nuestro terminal. (Puede usar como terminal, PowerShell, Terminal de Visual Studio Code o Cmder, recomiendo para este proyecto usar Cmder)



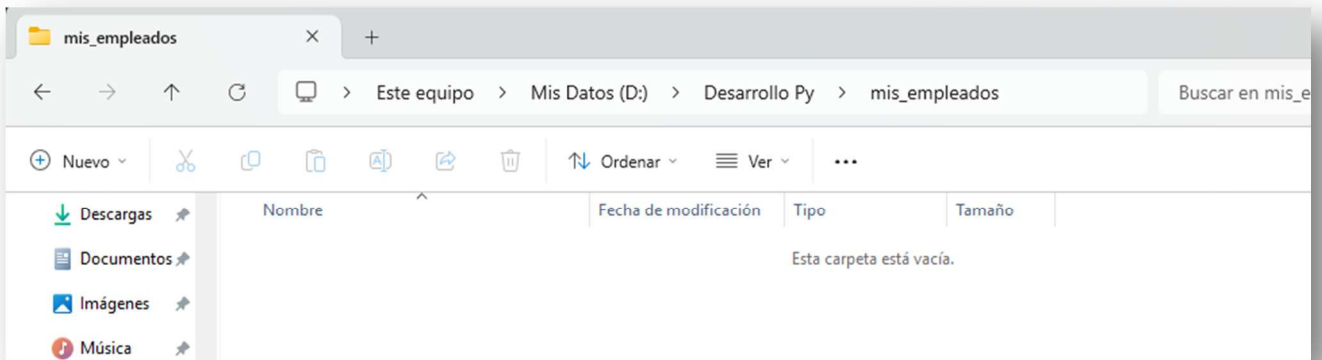
Creamos el entorno virtual : `py -m venv empleados_dj521` (empleados_dj521 es el nombre del entorno)



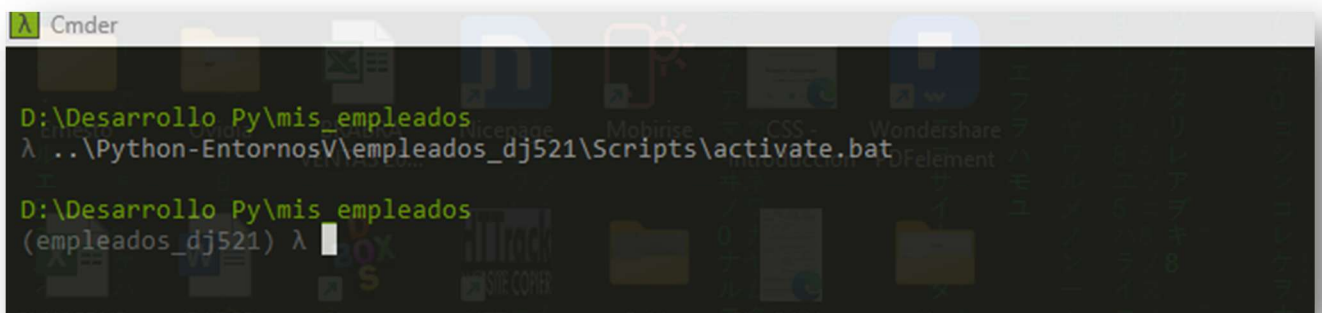
Automáticamente se crea la carpeta de nuestro entorno virtual



Regresamos al directorio donde crearemos nuestra aplicación, en este caso la carpeta “mis_empleados”, dentro de esa carpeta abrimos nuestro terminal.



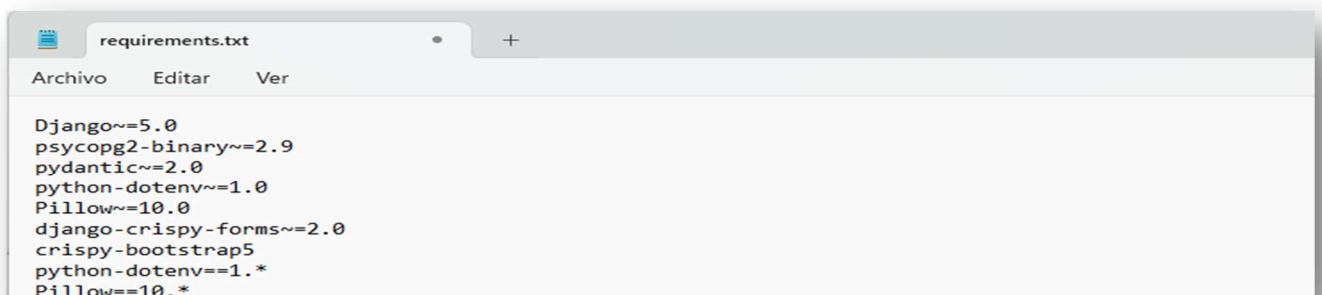
Activamos nuestro entorno virtual : `..\Python-Entornos\empleados_dj521\Scripts\activate.bat`



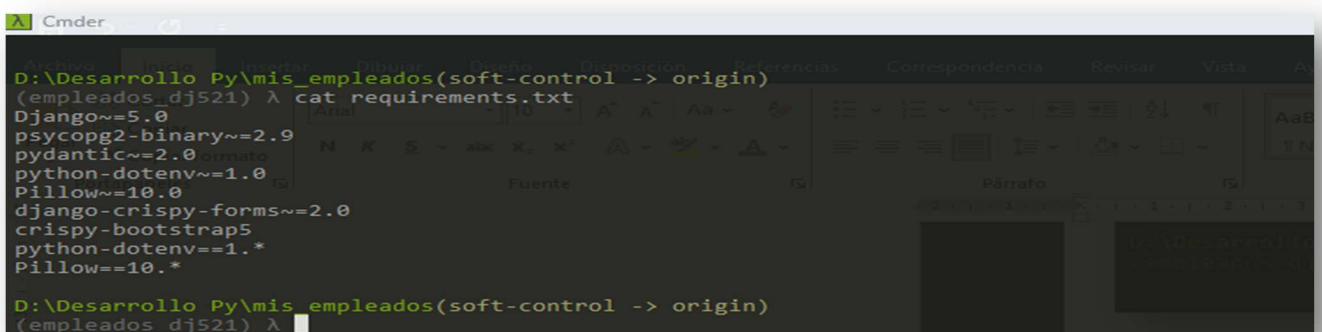
INSTALAR DEPENDENCIAS

Para instalar nuestras dependencias de manera automática, es recomendable crear un archivo y especificar cada una de las dependencias que se utilizara en nuestro proyecto. (generado por IA)

Utilice el editor de su preferencia y cree el archivo `requeriments.txt`, Guárdelo en la carpeta `mis_empleados`



Desde el terminal también puede verificar su contenido : `cat requirements.txt`



Y procedemos a instalar todas las dependencias : `pip install -r requirements.txt`

```
Cmder
D:\Desarrollo Py\empleados
(emp_employees_dj521) λ pip install -r requirements.txt
Collecting Django==4.2 (from -r requirements.txt (line 1))
  Obtaining dependency information for Django==4.2 from https://files.pythonhosted.org/packages/2c/4f/aeaa3098da18b625ed672f3da6d1cd94e188d1b2cc27c2c841b2f9666282/django-4.2.21-py3-none-any.whl.metadata (4.2 kB)
Collecting psycopg2-binary==2.9 (from -r requirements.txt (line 2))
  Obtaining dependency information for psycopg2-binary==2.9 from https://files.pythonhosted.org/packages/92/29/06261ea00e2dc1e22907dbbc483a1093665509eas586b29b8986a0e56733/psycopg2-binary-2.9.10-cp312-win_amd64.whl.metadata (5.0 kB)
Collecting pydantic==2.7 (from -r requirements.txt (line 3))
  Obtaining dependency information for pydantic==2.7 from https://files.pythonhosted.org/packages/e7/12/46b65f3534d099349e38ef6ec98b1a5a81f42536d17e0ba382c28c67ba67/pydantic-2.11.4-py3-none-any.whl.metadata (66 kB)
Collecting django-crispy-forms==2.0 (from -r requirements.txt (line 4))
  Obtaining dependency information for django-crispy-forms==2.0 from https://files.pythonhosted.org/packages/1d/ec/a25f81e56a674e63cf6c3dd8e3db1b3fccc238fec6d698504adc0cc61402/django_crispy_forms-2.0.1-py3-none-any.whl.metadata (5.2 kB)
Collecting asgiref<3.8.0, >=3.6.0 (from Django==4.2->-r requirements.txt (line 1))
  Obtaining dependency information for asgiref<3.8.0, >=3.6.0 from https://files.pythonhosted.org/packages/39/e5/893e87577be2012e0c20d99b58ad2e3051524b5b40cf50701e985a28b13e/asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from Django==4.2->-r requirements.txt (line 1))
  Obtaining dependency information for sqlparse>=0.3.1 from https://files.pythonhosted.org/packages/a9/5c/bfd6db0bf979426d405cc6e71ceeb8701b148b16c21d2dc3c261efc61c7b/sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from Django==4.2->-r requirements.txt (line 1))
  Obtaining dependency information for tzdata from https://files.pythonhosted.org/packages/5c/23/c7abc0ca0a1526a0774eca151daeb8de62ec457e77262b66b359c3c7679e/tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting annotated-types==0.6.0 (from pydantic==2.7->-r requirements.txt (line 3))
  Obtaining dependency information for annotated-types==0.6.0 from https://files.pythonhosted.org/packages/78/b6/6307fbef88db5ee7421e68d78a9f102e0da400bc5f5793fed3d0e34fb8/annotated_types-0.6.0-py3-none-any.whl.metadata (15 kB)
Using cached annotated-types==0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.33.2 (from pydantic==2.7->-r requirements.txt (line 3))
  Obtaining dependency information for pydantic-core==2.33.2 from https://files.pythonhosted.org/packages/43/e4/5479fecb3606c1368d496a825d8411e126133c41224c1e7238be58b87d7e/pydantic_core-2.33.2-cp312-win_amd64.whl.metadata (6.9 kB)
Collecting typing-inspection==0.4.0 (from pydantic==2.7->-r requirements.txt (line 3))
  Obtaining dependency information for typing-inspection==0.4.0 from https://files.pythonhosted.org/packages/43/e4/5479fecb3606c1368d496a825d8411e126133c41224c1e7238be58b87d7e/pydantic_core-2.33.2-cp312-win_amd64.whl.metadata (6.9 kB)
```

Verificamos que estén instaladas las dependencias: `pip freeze`

```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(emp_employees_dj521) λ pip freeze
annotated-types==0.7.0
asgiref==3.8.1
crispy-bootstrap5==2025.4
Django==4.2.21
django-crispy-forms==2.4
django-enviro==0.12.0
pillow==10.4.0
psycopg2-binary==2.9.10
pydantic==2.11.4
pydantic_core==2.33.2
python-dotenv==1.1.0
sqlparse==0.5.3
typing-inspection==0.4.0
typing_extensions==4.13.2
tzdata==2025.2

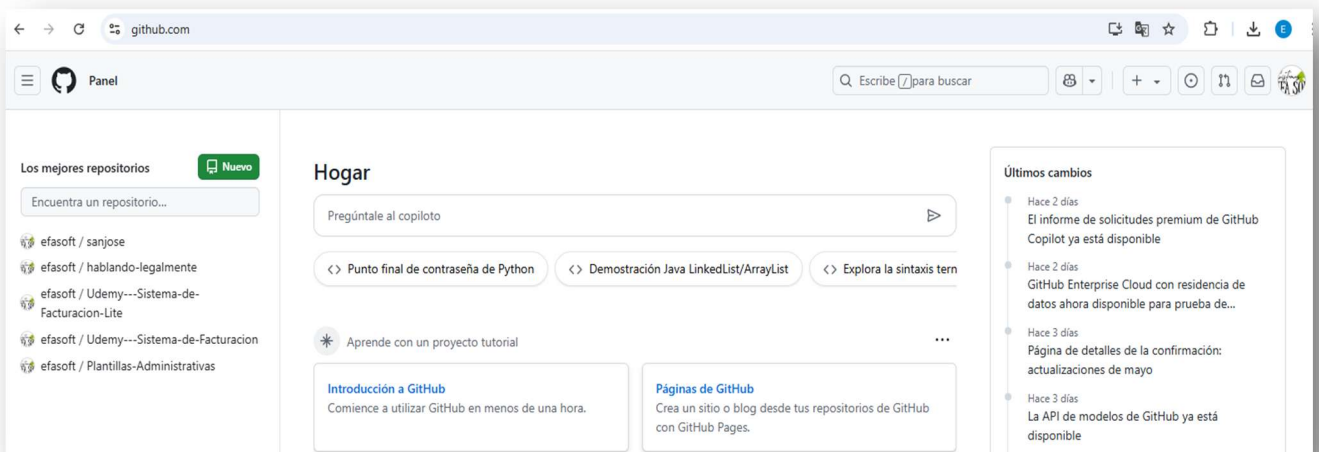
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(emp_employees_dj521) λ
```

REPOSITORIOS – CONTROL DE VERSIONES

Utilizaremos para el control de versiones la plataforma de **GitHub**. Para ello debemos crear nuestros **repositorios**

REPOSITORIO REMOTO

Ingresamos en nuestra cuenta de GitHub (<https://github.com/>) y creamos un nuevo repositorio, le pondremos como nombre **empleados**



REPOSITORIO LOCAL

Desde el terminal creamos nuestro repositorio local : `git init`

```
Cmder
D:\Desarrollo Py\mis_empleados
(empleados_dj521) λ git init
Initialized empty Git repository in D:/Desarrollo Py/mis_empleados/.git/

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ
```

Lo conectamos con nuestro repositorio remoto : `git remote add origin https://github.com/efasoft/empleados.git`

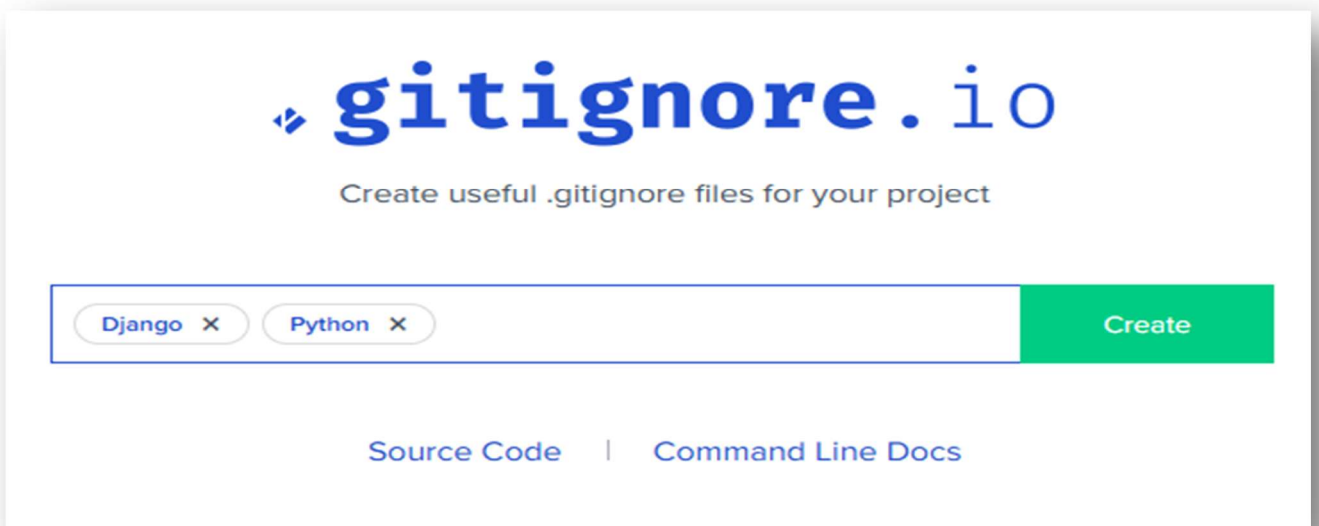
```
Cmder
D:\Desarrollo Py\mis_empleados
(empleados_dj521) λ git init
Initialized empty Git repository in D:/Desarrollo Py/mis_empleados/.git/

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git remote add origin https://github.com/efasoft/empleados.git

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ
```

Se recomienda crear el archivo `.gitignore`. El archivo `.gitignore` se utiliza en Git para especificar qué archivos o directorios deben ser ignorados por Git al realizar operaciones como añadir, confirmar o enviar cambios. Esto ayuda a mantener el repositorio limpio y evitar que se incluyan archivos innecesarios o confidenciales en la historia del proyecto.

Para crear el archivo puede ingresar a <https://www.toptal.com/developers/gitignore>



Indique el lenguaje o frameworks que va a utilizar y luego presione el botón

Create

. La web le mostrara todo el código generado.


```
← → ↻ toptal.com/developers/gitignore/api/django.python

# Created by https://www.toptal.com/developers/gitignore/api/django,python
# Edit at https://www.toptal.com/developers/gitignore?templates=django,python

### Django ###
*.log
*.pot
*.pyc
__pycache__/
local_settings.py
db.sqlite3
db.sqlite3-journal
media

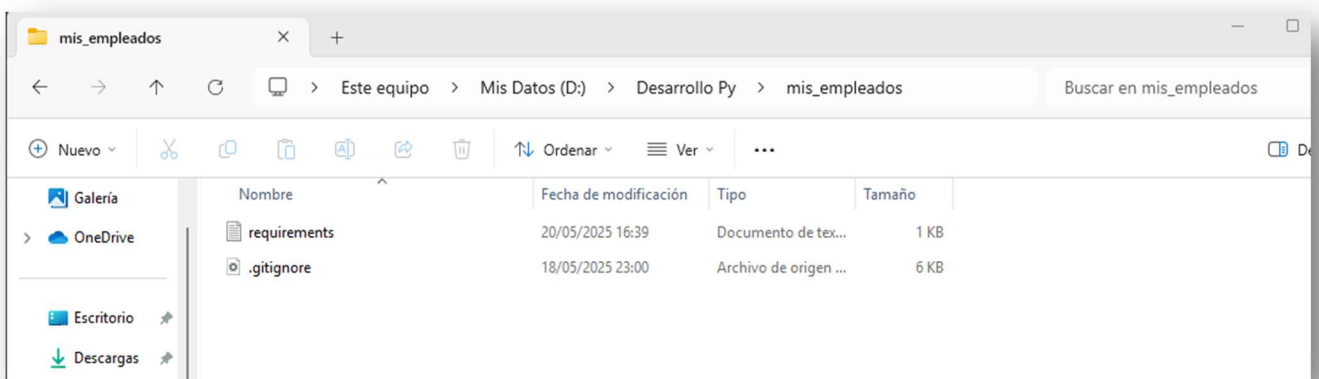
# If your build process includes running collectstatic, then you probably don't need or want to include staticfiles/
# in your Git repository. Update and uncommment the following line accordingly.
# <django-project-name>/staticfiles/

### Django.Python Stack ###
# Byte-compiled / optimized / DLL files
*.py[cod]
*$py.class

# C extensions
*.so
```

Abra su editor de texto preferido y copie todo el contenido, luego guarde el archivo dentro de la carpeta de su proyecto. (en este caso la carpeta “mis_empleados”). Recuerde el nombre del archivo debe ser .gitignore

Comprobamos



Una vez creado el archivo .gitignore, realizamos la primera actualización de nuestros repositorios.

Para agregar todos los archivos que han sido creados o modificados : **git add** .

Identificamos nuestra actualización: **git commit -m "Formulario de Empleados con Django y Pydantic"**

Cada vez que haga un **commit** debe poner entre comillas un comentario.

Actualizamos nuestros repositorios: **git push -u origin soft-control**

```
Cmder

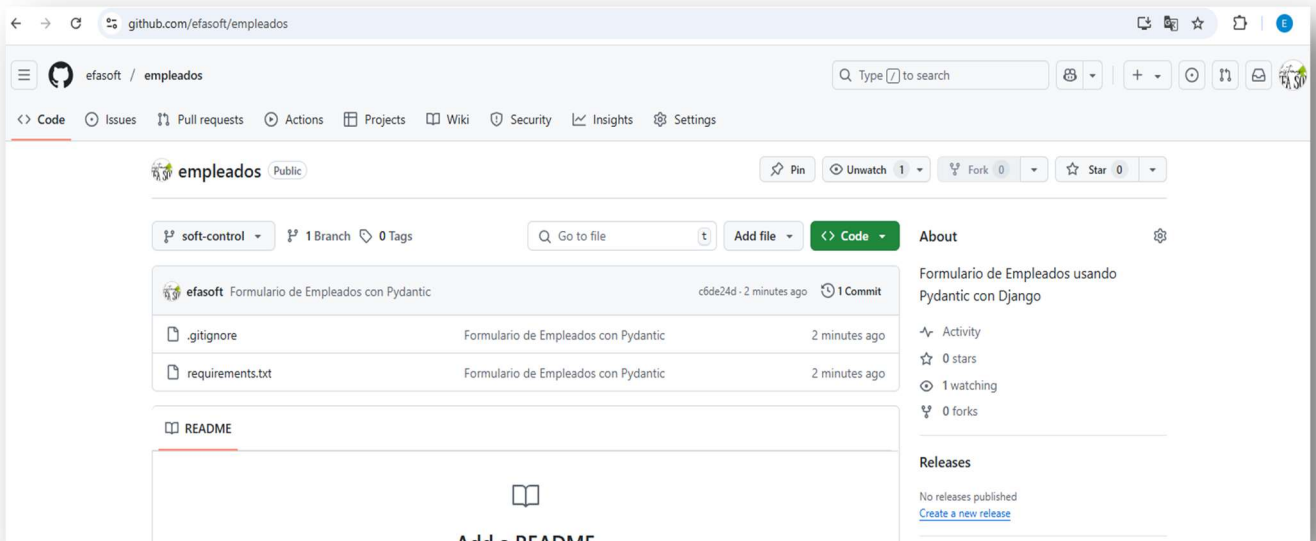
D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git add .
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git commit -m "Formulario de Empleados con Django y Pydantic"
[soft-control (root-commit) 1aa2b9a] Formulario de Empleados con Django y Pydantic
2 files changed, 273 insertions(+)
create mode 100644 .gitignore
create mode 100644 requirements.txt

D:\Desarrollo Py\mis_empleados(soft-control)
(empleados_dj521) λ git push -u origin soft-control
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.17 KiB | 2.17 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/efasoft/empleados.git
 * [new branch]      soft-control -> soft-control
branch 'soft-control' set up to track 'origin/soft-control'.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

Comprobamos que nuestro repositorio remoto este actualizado. Recomendamos que periódicamente se actualicen los repositorios.



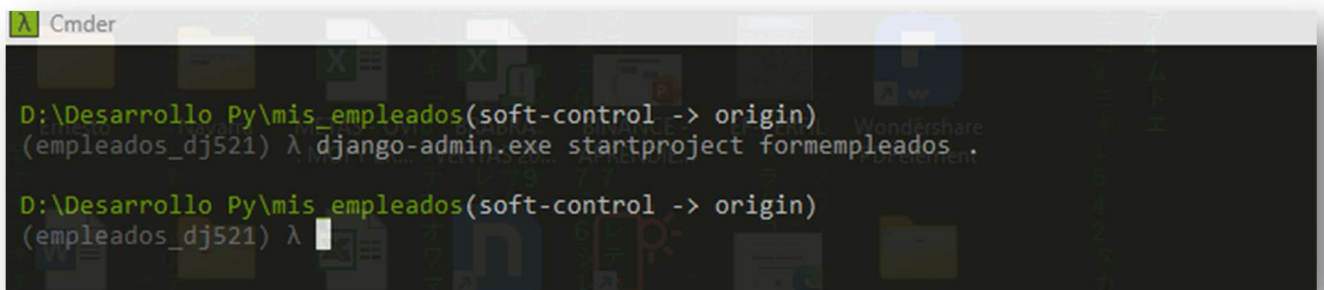
CONFIGURACIÓN INICIAL DEL PROYECTO

CREACIÓN DEL PROYECTO

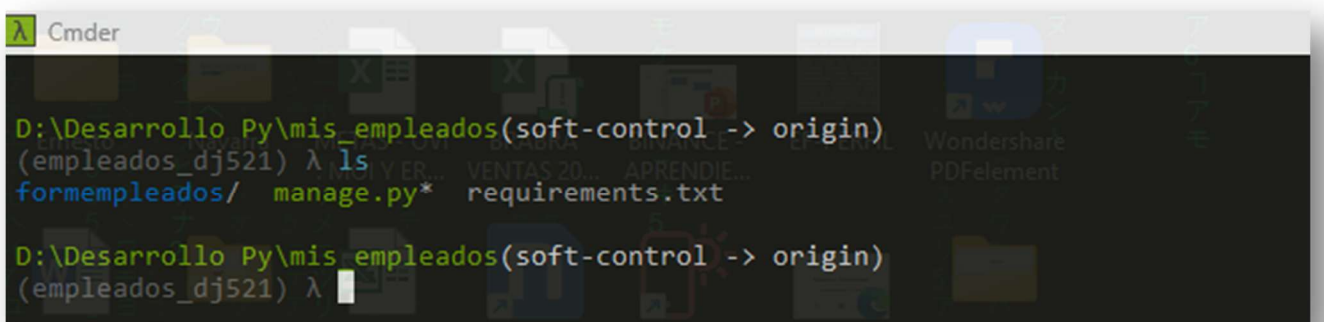
Abrimos el terminal dentro de nuestra carpeta de desarrollo y ejecutamos el siguiente comando :

`django-admin.exe startproject formempleados .`

`formempleados .` es el nombre del proyecto, después del nombre deje un espacio y coloque un punto, esto es para que no se cree una carpeta adicional



Verificamos que este creado el proyecto : `ls`

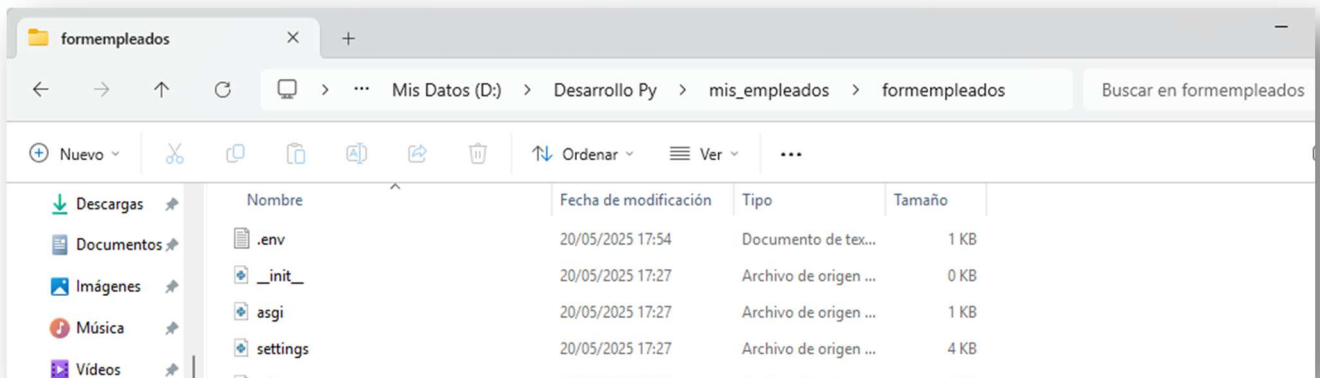


Antes de continuar, asegúrate de tener **PostgreSQL** instalado y un usuario y base de datos creados.

Usando nuestro editor preferido **creamos** el archivo **.env** (Este archivo debe estar en la raíz de tu proyecto **form_empleados**)

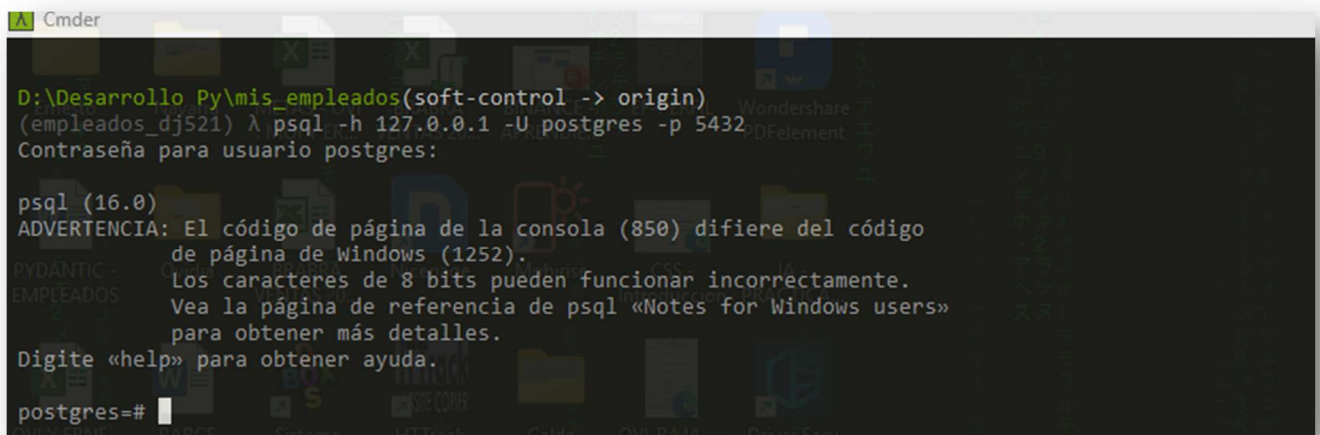
```
DJANGO_SECRET_KEY = keep the secret key used in production secret! (settings.py)
DB_NAME=your_database
DB_USER=your_postgres_user
DB_PASSWORD=your_postgres_password
DB_HOST=localhost
DB_PORT=5432
```

No olvides modificar y poner los valores reales para acceder a tu Base de Datos



CREAR LA BASE DE DATOS

Desde el terminal nos **conectamos** a PostgreSQL : **psql -h 127.0.0.1 -U postgres -p 5432**



Explicación:

1. psql:

- psql es la **interfaz de línea de comandos interactiva para PostgreSQL**. Es una aplicación cliente que te permite ejecutar consultas SQL, administrar bases de datos y realizar otras tareas directamente desde tu terminal. Piensa en él como la "consola" o el "terminal" específico para interactuar con tu servidor PostgreSQL.

2. -h 127.0.0.1:

- h es una opción (flag) que especifica el **nombre de host o la dirección IP del servidor PostgreSQL** al que quieres conectarte.

- 127.0.0.1 es la **dirección IP de "localhost"**. Esto significa que estás intentando conectarte a un servidor PostgreSQL que se está ejecutando en la **misma máquina** donde estás ejecutando el comando psql.

3. -U postgres:

- -U es una opción que especifica el **nombre de usuario** con el que quieres autenticarte en el servidor PostgreSQL.
- postgres es el **nombre de usuario predeterminado** que se crea durante la instalación de PostgreSQL. A menudo se utiliza para tareas de administración y superusuario.

4. -p 5432:

- -p es una opción que especifica el **número de puerto TCP** en el que el servidor PostgreSQL está escuchando las conexiones entrantes.
- 5432 es el **puerto predeterminado** que PostgreSQL utiliza para escuchar las conexiones. A menos que hayas configurado tu servidor PostgreSQL para usar un puerto diferente, este será el puerto correcto.

Creamos la Base de Datos en PostgreSQL : `create database db_empleados_521;`

```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ psql -h 127.0.0.1 -U postgres -p 5432
Contraseña para usuario postgres:

psql (16.0)
ADVERTENCIA: El código de página de la consola (850) difiere del código
de página de Windows (1252).
Los caracteres de 8 bits pueden funcionar incorrectamente.
Vea la página de referencia de psql «Notes for Windows users»
para obtener más detalles.
Digite «help» para obtener ayuda.
postgres=# create database db_empleados_521;
CREATE DATABASE
postgres=#

```

Comprobamos la Base de Datos en PostgreSQL: `\l`

```

postgres=# \l

```

Nombre	Dueño	Codificación	Proveedor de locale	Collate	Ctype	configuración ICU
db_djcmpfc_42	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
db_empleados_521	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
db_sanjose_42	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
dbprueba	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
postgres	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
template0	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	
template1	postgres	UTF8	libc	Spanish_Spain.1252	Spanish_Spain.1252	

```

(7 filas)

postgres=#

```

APP DEL PROYECTO (empleados y usuarios)

Abrimos el terminal dentro de nuestra carpeta de desarrollo y creamos las apps

`py manage.py startapp empleados`
`py manage.py startapp usuarios`

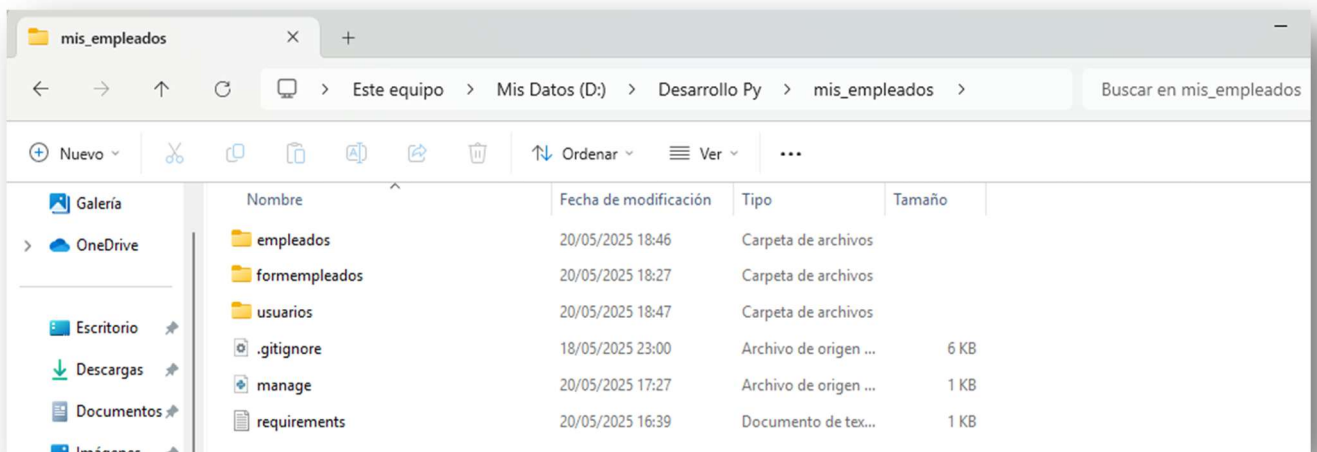
```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py startapp empleados

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py startapp usuarios

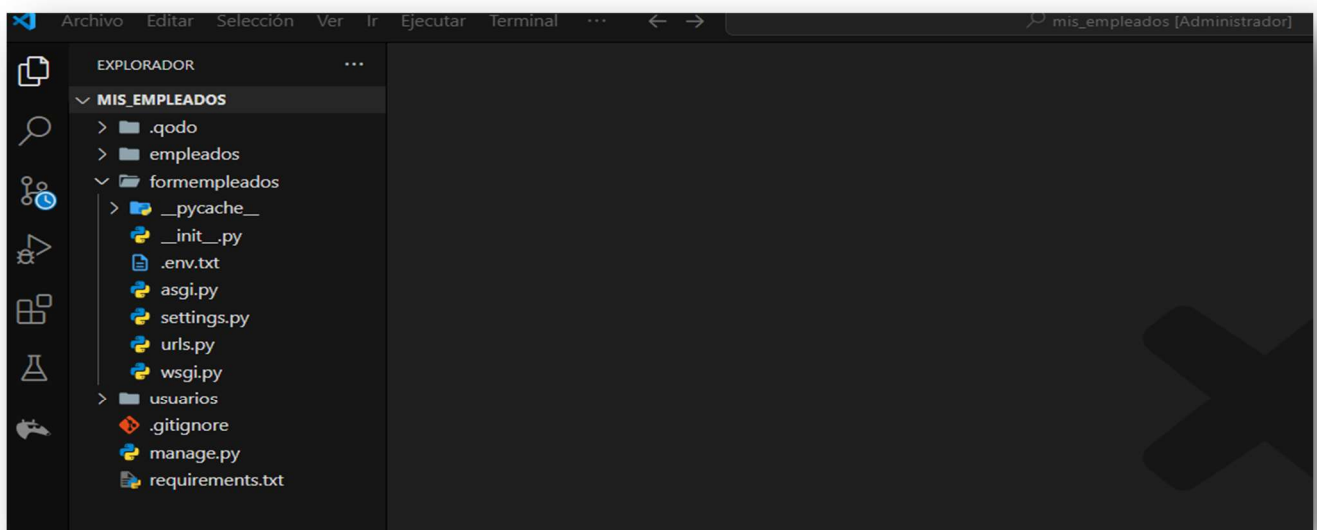
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ

```

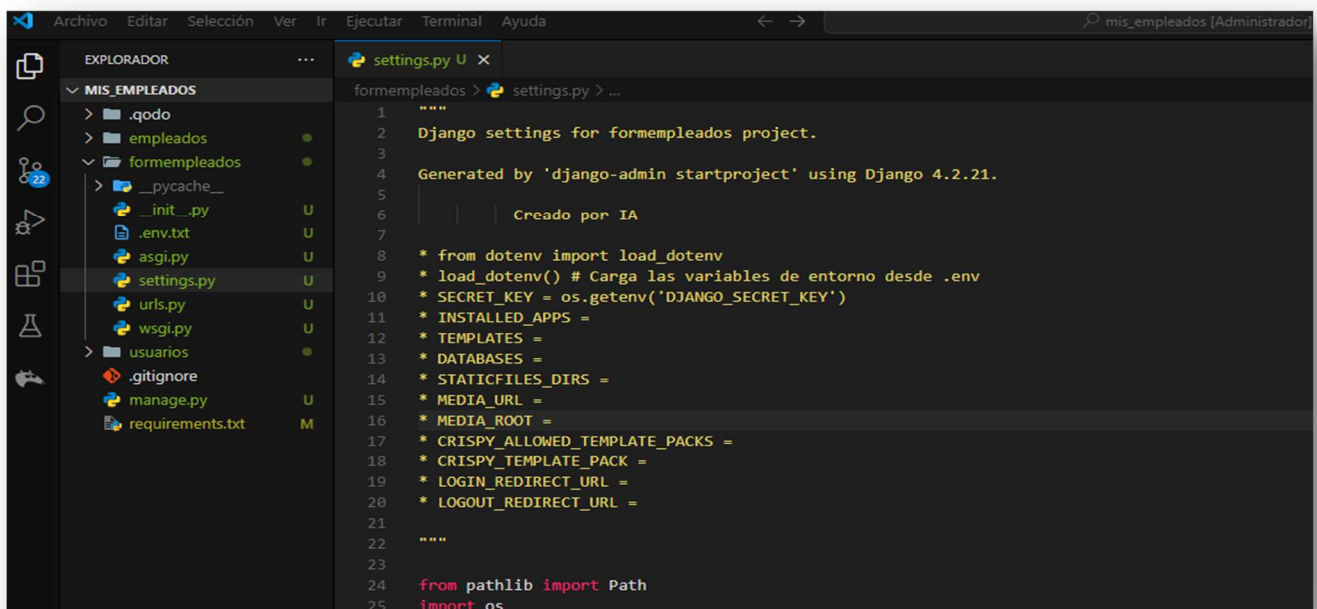
DESARROLLO DEL PROYECTO

Desde nuestro terminal abrimos Visual Studio Code : `code -r .`



Observaremos que se crearon las carpetas `formempleados`, `empleados`, `usuarios` y dentro de ellas los archivos iniciales del proyecto.

CONFIGURACIÓN PRINCIPAL `settings.py` (generado por IA)



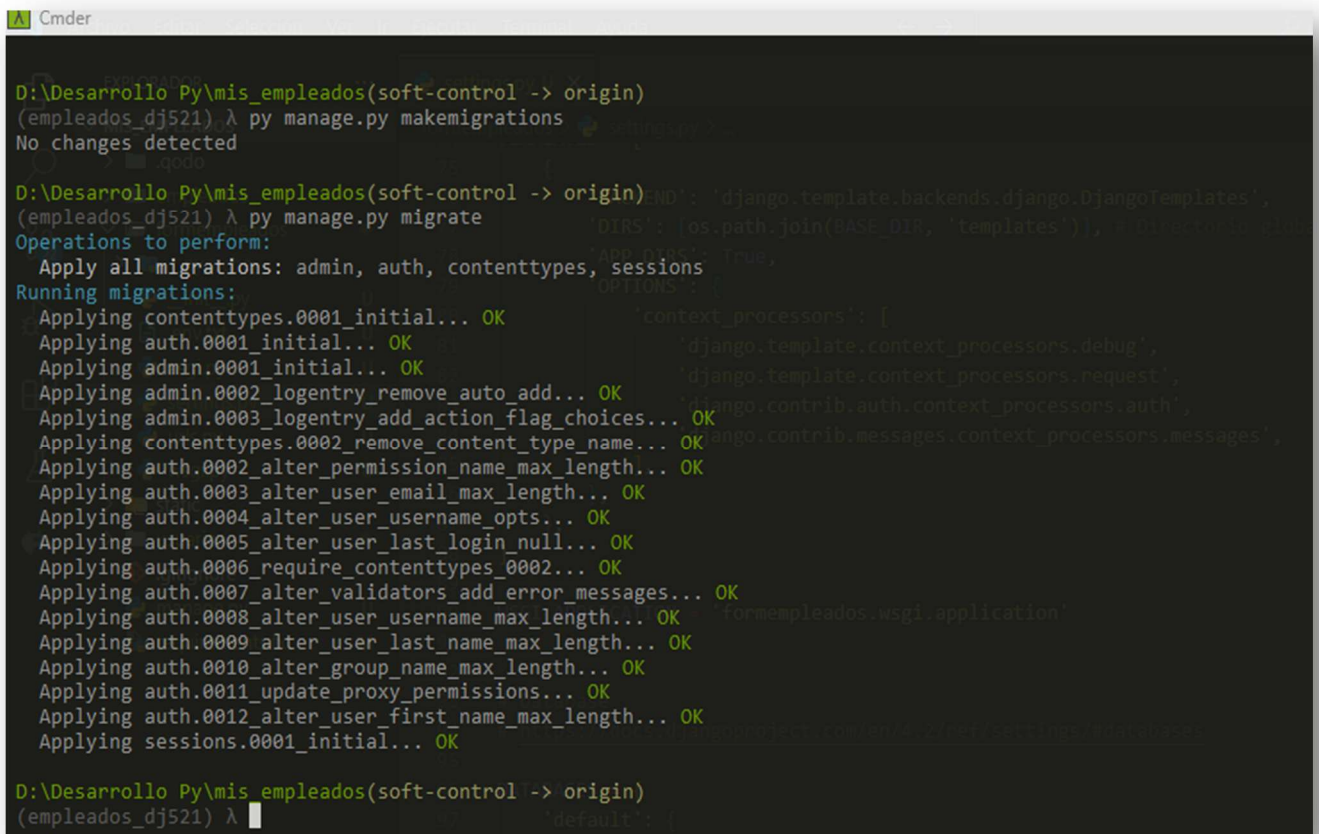
Explicación:

- **SECRET_KEY:** Una clave secreta para la seguridad de tu aplicación. ¡Mantenla segura!
- **DEBUG:** Activa o desactiva el modo de depuración. En producción, debe ser False.
- **ALLOWED_HOSTS:** Lista de hosts permitidos para tu aplicación.
- **INSTALLED_APPS:** Las aplicaciones Django que utilizará tu proyecto. Hemos añadido empleados y usuarios.
- **MIDDLEWARE:** Componentes que procesan las peticiones y respuestas.
- **ROOT_URLCONF:** El archivo de URLs principal del proyecto.
- **TEMPLATES:** Configuración de los motores de plantillas.
- **DATABASES:** Configuración de la base de datos PostgreSQL. Usamos environ para leer las variables de entorno.
- **AUTH_PASSWORD_VALIDATORS:** Validadores de contraseñas por defecto de Django.
- **LANGUAGE_CODE y TIME_ZONE:** Configuración de idioma y zona horaria para España.
- **STATIC_URL y STATICFILES_DIRS:** Configuración de archivos estáticos (CSS, JS, imágenes).
- **DEFAULT_AUTO_FIELD:** El tipo de campo automático primario por defecto.
- **MESSAGE_TAGS:** Configuración para que los mensajes de Django se mapeen a los estilos de Bootstrap (para SweetAlert2).
- **AUTH_USER_MODEL:** Indica que usaremos nuestro modelo de usuario personalizado.

MIGRACIONES INICIALES DEL PROYECTO (generado por IA)

Desde nuestro terminal ejecutamos:

```
py manage.py makemigrations
py manage.py migrate
```



```
Cmder

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py makemigrations
No changes detected

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ
```

Las migraciones iniciales de **admin**, **auth**, **content types** y **sessions** en Django son un conjunto de tablas y datos que se crean automáticamente al iniciar un proyecto Django. Estas migraciones son fundamentales para el funcionamiento básico del framework, gestionando la interfaz de administración, la autenticación de usuarios, los tipos de contenido y las sesiones de usuario, respectivamente.

Detalle de cada migración:

- **admin:**
Esta migración crea la interfaz de administración de Django, permitiendo a los usuarios gestionar los modelos y datos de la base de datos a través de un panel web.
- **auth:**
Esta migración proporciona la funcionalidad de autenticación y autorización de usuarios. Permite crear usuarios, grupos y permisos para controlar el acceso a diferentes partes de la aplicación.

- **content types:**
Esta migración gestiona la clasificación de los tipos de contenido. Permite identificar las diferentes clases de modelos que se pueden utilizar en Django.
- **sessions:**
Esta migración implementa la gestión de sesiones de usuario. Almacena información temporal sobre la actividad del usuario en la aplicación.

En resumen: estas migraciones iniciales son el punto de partida para el desarrollo de una aplicación Django, proporcionando la base necesaria para la gestión de usuarios, la administración de la aplicación y la gestión de datos. Son creadas automáticamente por Django al ejecutar el comando **makemigrations** por primera vez y se aplican con el comando **migrate**.

Comprobamos la Migración Inicial : **py manage.py showmigrations**

```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py showmigrations
admin
[X] 0001_initial
[X] 0002_logentry_remove_auto_add
[X] 0003_logentry_add_action_flag_choices
auth
[X] 0001_initial
[X] 0002_alter_permission_name_max_length
[X] 0003_alter_user_email_max_length
[X] 0004_alter_user_username_opts
[X] 0005_alter_user_last_login_null
[X] 0006_require_contenttypes_0002
[X] 0007_alter_validators_add_error_messages
[X] 0008_alter_user_username_max_length
[X] 0009_alter_user_last_name_max_length
[X] 0010_alter_group_name_max_length
[X] 0011_update_proxy_permissions
[X] 0012_alter_user_first_name_max_length
contenttypes
[X] 0001_initial
[X] 0002_remove_content_type_name
empleados
(no migrations)
sessions
[X] 0001_initial
usuarios
(no migrations)

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ

```

En este punto del proyecto no hemos definido los datos (en Python llamados “modelos”) de las app de **empleados** y **usuarios**, es por eso que al realizar la primera migración muestra en rojo un error. Es importante recordar que cada vez que se agregue o modifique un **modelo** se debe realizar el procedimiento de migración.

SUPERUSUARIO – PANEL DE ADMINISTRACIÓN (generado por IA)

Una vez que levantemos nuestro servidor de desarrollo, podremos ingresar al Panel de Administración de Django, usando este **superusuario**. Para ello desde nuestro terminal **ejecutamos**:

py manage.py createsuperuser

```

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py createsuperuser
Nombre de usuario (leave blank to use 'ernesto'):
Dirección de correo electrónico: efasoft@hotmail.com
Password:
Password (again):
Superuser created successfully.

D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ

```

Para este proyecto el **password** será: **empleados2025**

SERVIDOR DE DESARROLLO (generado por IA)

Levantamos nuestro servidor de desarrollo : `py manage.py runserver`

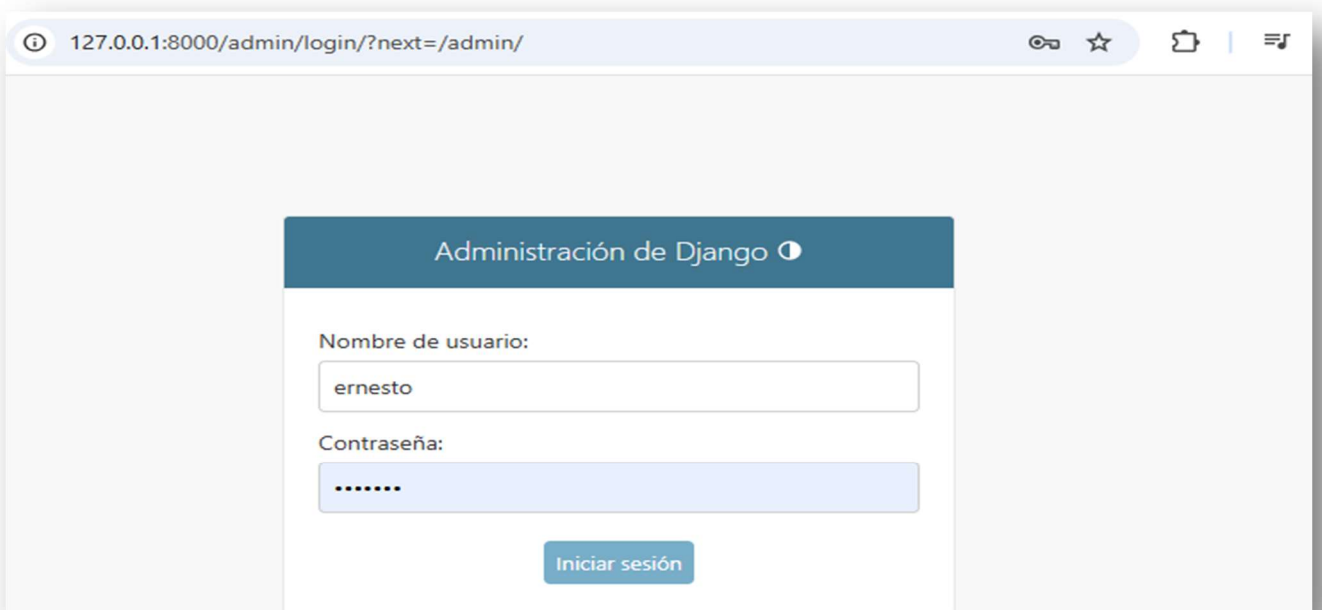
```
Cmder
D:\Desarrollo Py\mis_empleados(soft-control -> origin)
(empleados_dj521) λ py manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 22, 2025 - 19:44:23
Django version 4.2.21, using settings 'formempleados.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Abre tu navegador preferido e ingresa a <http://127.0.0.1:8000/>

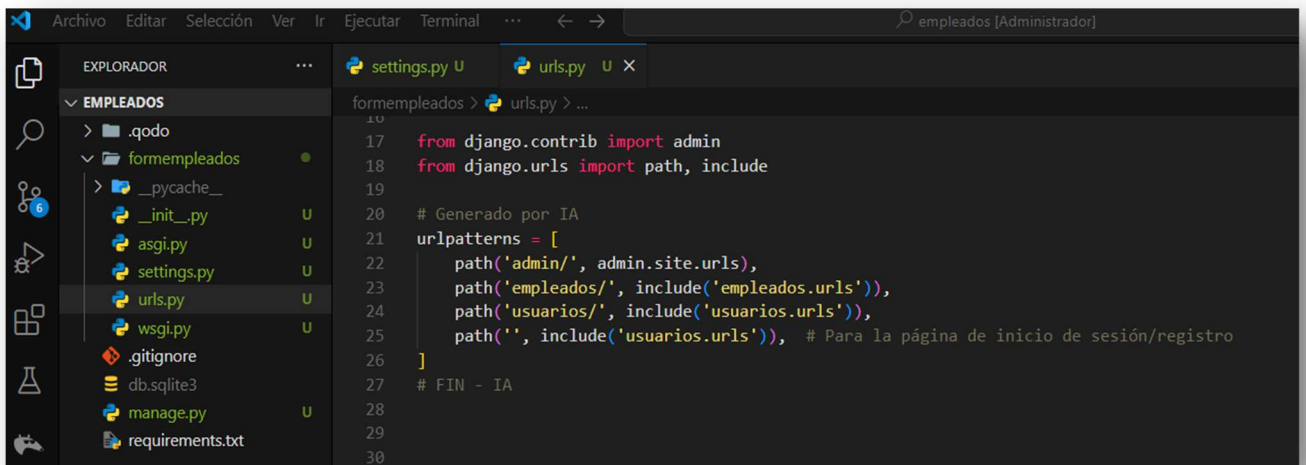


Para ingresar al Panel de Administración, ingresa a <http://127.0.0.1:8000/admin/>.





URLS PRINCIPAL `urls.py` (generado por IA)



Explicación:

- Define las rutas URL principales del proyecto, incluyendo las URLs de la aplicación `empleados` y `usuarios`.

CREAMOS LA BASE DE DATOS DE NUESTRO PROYECTO

SOLICIONAR ESTOS ERRORES

Verificamos [Conexión](#) con la Base de Datos en PostgreSQL: `py manage.py dbshell`

Lectora del archivo `.env`



